

DISCRETE STRUCTURE

Lesson-wise Notes with Python Practice

For: BS Computer Science — 1st Semester

Duration: 16 Weeks / 32 Lectures

Teaching philosophy

Every lesson follows Theory → Mathematical Example → Computer Science Connection → Python Implementation → Worked Example → Practice. Python is used as a tool to explore and implement discrete-mathematics concepts, not as a replacement for mathematical reasoning.

Prepared by Abdul Salam

Course Alignment

These notes are designed around the HEC Computing Disciplines curriculum course Discrete Structure, 3 credit hours, with no prerequisite. The HEC outline includes mathematical reasoning, propositional and predicate logic, rules of inference, induction, proof techniques, sets, relations, equivalence relations and partitions, partial orderings, recurrence relations, functions and mappings, number theory, sequences and series, counting, inclusion-exclusion, pigeonhole principle, permutations and combinations, algorithms, searching and sorting, graph theory, planar graphs, graph coloring, graph algorithms, Euler and Hamiltonian concepts, rooted trees and traversals.

The 32-lecture plan compresses the HEC content into 16 teaching weeks and integrates Python into each topic. Predicate logic and partial orderings are treated as focused subtopics inside the logic/relations lessons so the outline remains covered without over-fragmenting a first-semester course.

Important: Python can verify finite examples, but finite testing is not a mathematical proof for all possible values.

16-Week Course Map

Week	Lectures	Topics
1	1–2	Introduction to Discrete Structures and Python; Propositions, Statements and Truth Values
2	3–4	Logical Operators and Truth Tables; Logical Equivalence, Tautology and Contradiction
3	5–6	Rules of Inference and Arguments; Methods of Proof
4	7–8	Mathematical Induction; Recursion and Recursive Algorithms
5	9–10	Sets: Definitions, Representation and Types; Set Operations
6	11–12	Cartesian Product and Power Set; Sets, Venn Diagrams and Counting Applications
7	13–14	Relations: Definition, Representation and Properties; Equivalence Relations and Partitions
8	15–16	Functions and Mappings; Composition, Inverse and Recursive Functions
9	17–18	Sequences and Series; Recurrence Relations
10	19–20	Counting Principles: Sum and Product Rules; Permutations and Combinations
11	21–22	Binomial Coefficients and Binomial Theorem; Inclusion–Exclusion Principle
12	23–24	Pigeonhole Principle; Number Theory: Divisibility, Primes and GCD
13	25–26	Modular Arithmetic and Applications; Algorithms: Searching and Sorting
14	27–28	Graph Theory: Basic Concepts; Graph Representation, Paths and Traversal
15	29–30	Euler and Hamiltonian Graphs; Graph Coloring and Planar Graphs
16	31–32	Trees, Rooted Trees and Traversals; Comprehensive Review and Integrated Python Applications

Lesson 1

1. Introduction to Discrete Structures and Python

Learning Objectives

Understand what discrete mathematics is, why it matters in computing, and learn the small amount of Python needed throughout the course.

1. Theoretical Concept

- Discrete mathematics studies objects that are countable or separable rather than continuously varying.
- Computer science uses discrete objects such as bits, integers, strings, sets, graphs, trees, logical statements and finite algorithms.
- A discrete structure is a mathematical structure used to describe relationships among discrete objects.
- This course connects mathematical reasoning with computational experiments in Python.

2. Computer Science Connection

Discrete structures appear in programming languages, databases, cybersecurity, algorithms, networking and artificial intelligence.

3. Python Implementation

```
# Basic Python used in this course
a = 10
b = 3
print("Sum =", a + b)
print("Product =", a * b)
print("a > b:", a > b)
print("a is even:", a % 2 == 0)
```

4. Worked Example / Classroom Explanation

The main idea is not to replace mathematical reasoning with programming. Python is used to verify examples, automate repetitive calculations, explore patterns and connect mathematics to computing.

5. Practice Questions

1. Write a Python program that calculates the sum, difference and product of two integers.
2. Use % to determine whether 37 is even or odd.
3. Explain in your own words why discrete mathematics is important in computer science.
4. Modify the code so the user enters the two numbers with input().

6. Lesson Summary

Students learned the central ideas of Introduction to Discrete Structures and Python, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 2

2. Propositions, Statements and Truth Values

Learning Objectives

Identify propositions and represent their truth values in Python.

1. Theoretical Concept

- A proposition is a declarative statement that is either true or false, but not both.
- Questions, commands and open sentences containing an unknown variable are not propositions until the variable is specified.
- Truth values are True and False in Python.
- A proposition can be represented by a Boolean variable.

2. Computer Science Connection

Boolean expressions control if-statements, loops, validation rules and digital circuits.

3. Python Implementation

```
p = True
q = False

print(p)
print(q)
print(type(p))
x = 7
print(x > 5)
```

4. Worked Example / Classroom Explanation

Examples: '5 is prime' is a proposition; 'Close the door' is not a proposition; 'x > 5' is an open statement until x is known.

5. Practice Questions

1. Classify five given statements as propositions or non-propositions.
2. Create Boolean variables p, q and r in Python.
3. Write a program that prints whether n is greater than 10.
4. Explain the difference between a proposition and an open statement.

6. Lesson Summary

Students learned the central ideas of Propositions, Statements and Truth Values, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 3

3. Logical Operators and Truth Tables

Learning Objectives

Construct truth tables for NOT, AND, OR, XOR and implication, and reproduce them in Python.

1. Theoretical Concept

- Negation $\neg p$ reverses the truth value of p .
- Conjunction $p \wedge q$ is true only when both are true.
- Disjunction $p \vee q$ is true when at least one is true.
- Exclusive OR is true when exactly one input is true.
- Implication $p \rightarrow q$ is false only when p is true and q is false.

Truth-table size

For n propositions there are 2^n possible rows. Two propositions give 4 rows and three propositions give 8 rows.

2. Computer Science Connection

Logical operators directly correspond to conditions used in software and digital systems.

3. Python Implementation

```
def implication(p, q):  
    return (not p) or q  
  
values = [True, False]  
for p in values:  
    for q in values:  
        print(p, q, p and q, p or q, implication(p, q))
```

4. Worked Example / Classroom Explanation

Truth tables are useful in digital logic, program conditions, formal verification and circuit design.

5. Practice Questions

1. Draw truth tables for $\neg p$, pq , $p \vee q$ and $p \rightarrow q$.
2. Write Python code for XOR without using $\wedge$.
3. Generate the complete truth table of $(p \wedge q) \vee (\neg p)$.
4. Identify the only false row of $p \rightarrow q$.

6. Lesson Summary

Students learned the central ideas of Logical Operators and Truth Tables, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 4

4. Logical Equivalence, Tautology and Contradiction

Learning Objectives

Test compound propositions for equivalence and identify tautologies and contradictions.

1. Theoretical Concept

- Two propositions are logically equivalent if they have the same truth value in every possible case.
- A tautology is always true.
- A contradiction is always false.
- A contingency is true in some cases and false in others.

2. Computer Science Connection

Equivalence checking is related to simplification and verification of Boolean expressions.

3. Python Implementation

```
from itertools import product

for p, q in product([True, False], repeat=2):
    left = not (p and q)
    right = (not p) or (not q)
    print(p, q, left == right)
```

4. Worked Example / Classroom Explanation

Python can enumerate all Boolean combinations and compare truth values automatically.

5. Practice Questions

1. Use a truth table to verify De Morgan's law.
2. Write a program to test whether p or not p is a tautology.
3. Test whether p and not p is a contradiction.
4. Compare two expressions and report whether they are equivalent.

6. Lesson Summary

Students learned the central ideas of Logical Equivalence, Tautology and Contradiction, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 5

5. Rules of Inference and Arguments

Learning Objectives

Understand premises, conclusions and common inference rules and test simple arguments computationally.

1. Theoretical Concept

- An argument consists of premises and a conclusion.
- Modus ponens: $p \rightarrow q$, p , therefore q .
- Modus tollens: $p \rightarrow q$, $\neg q$, therefore $\neg p$.
- Hypothetical syllogism: $p \rightarrow q$ and $q \rightarrow r$ imply $p \rightarrow r$.
- An argument is valid when no case exists in which all premises are true and the conclusion is false.

Predicate-logic connection

A predicate $P(x)$ becomes a proposition when x is specified. Quantifiers use ('for every') and ('there exists'). Python can test finite domains, while universal claims require reasoning.

2. Computer Science Connection

Inference is central to rule-based systems, program verification and automated reasoning.

3. Python Implementation

```
from itertools import product

def valid_argument(premises, conclusion):
    for row in product([True, False], repeat=len(premises)):
        env = dict(zip("pqrstu", row))
        if all(eval(x, {}, env) for x in premises):
            if not eval(conclusion, {}, env):
                return False, env
    return True, None

print(valid_argument(["(not p) or q", "p"], "q"))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Explain modus ponens with a real-life computing example.
2. Determine whether $p \rightarrow q$, q therefore p is valid.
3. Write a Python checker for a two-premise argument.
4. Find a counterexample if an argument is invalid.

6. Lesson Summary

Students learned the central ideas of Rules of Inference and Arguments, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 6

6. Methods of Proof

Learning Objectives

Distinguish direct proof, proof by contraposition, contradiction and implication.

1. Theoretical Concept

- A direct proof starts from the assumptions and derives the conclusion.
- For $p \rightarrow q$, the contrapositive is $\neg q \rightarrow \neg p$ and is logically equivalent to the original implication.
- Proof by contradiction assumes the conclusion is false and derives an impossibility.
- A proof by cases divides the problem into exhaustive cases.

2. Computer Science Connection

Proof methods help establish correctness of algorithms and mathematical claims.

3. Python Implementation

```
# Experimental check: if n is even, n^2 is even
for n in range(-10, 11):
    if n % 2 == 0:
        assert (n*n) % 2 == 0
print("All tested cases passed.")
```

4. Worked Example / Classroom Explanation

Python can check many finite instances, but finite testing is not a mathematical proof for all possible integers.

5. Practice Questions

1. Give a direct proof that the sum of two even integers is even.
2. Write the contrapositive of 'If n is divisible by 4, then n is even.'
3. Give a contradiction proof that an integer cannot be both even and odd.
4. Explain why a Python test over 1,000 values is not a proof for all integers.

6. Lesson Summary

Students learned the central ideas of Methods of Proof, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 7

7. Mathematical Induction

Learning Objectives

Use mathematical induction and connect it with automated verification.

1. Theoretical Concept

- Induction has two essential parts: base case and inductive step.
- If $P(k)$ implies $P(k+1)$, and $P(1)$ is true, then $P(n)$ is true for every integer $n \geq 1$.
- The induction hypothesis is the assumption that $P(k)$ is true for an arbitrary k in the inductive step.

2. Computer Science Connection

Induction is often used to prove statements about algorithms, recursive structures and running-time formulas.

3. Python Implementation

```
def lhs(n):  
    return sum(range(1, n+1))  
  
def rhs(n):  
    return n * (n + 1) // 2  
  
for n in range(1, 101):  
    assert lhs(n) == rhs(n)  
  
print("Formula verified for n = 1,...,100")
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Prove $1+2+\dots+n = n(n+1)/2$ by induction.
2. Verify the formula computationally for $n=1$ to 1000.
3. Prove $1+3+5+\dots+(2n-1)=n^2$.
4. Explain base case and inductive step in simple words.

6. Lesson Summary

Students learned the central ideas of Mathematical Induction, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 8

8. Recursion and Recursive Algorithms

Learning Objectives

Understand recursive definitions and implement simple recursive functions.

1. Theoretical Concept

- A recursive function calls itself on a smaller version of the same problem.
- Every useful recursion needs a base case and a recursive case.
- Recursion appears in factorial, Fibonacci numbers, tree traversal and divide-and-conquer algorithms.

2. Computer Science Connection

Recursion is a natural way to process trees and divide-and-conquer problems.

3. Python Implementation

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(factorial(5))
print([fibonacci(i) for i in range(8)])
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Write a recursive function for sum $1+\dots+n$.
2. Trace factorial(4) step by step.
3. Write recursive Fibonacci and compare it with an iterative version.
4. Identify the base case in each function.

6. Lesson Summary

Students learned the central ideas of Recursion and Recursive Algorithms, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 9

9. Sets: Definitions, Representation and Types

Learning Objectives

Represent sets mathematically and in Python and identify membership and subset relationships.

1. Theoretical Concept

- A set is a well-defined collection of distinct objects.
- Roster notation lists elements; set-builder notation describes a rule.
- The empty set is $\emptyset$. A singleton has exactly one element.
- $A \subseteq B$ means every element of A is also an element of B.

2. Computer Science Connection

Sets model collections such as users, database records, permissions and unique items.

3. Python Implementation

```
A = {1, 2, 3, 4, 5}
B = {2, 4}

print(3 in A)
print(B.issubset(A))
print(len(A))
print(A)

# set-builder style example: even numbers from 0 to 10
E = {x for x in range(11) if x % 2 == 0}
print(E)
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Create a set of the first 10 natural numbers.
2. Test whether $\{2,4\}$ is a subset of $\{1,2,3,4\}$.
3. Create the set of odd integers from 1 to 19 using a set comprehension.
4. Explain why duplicate elements do not change a set.

6. Lesson Summary

Students learned the central ideas of Sets: Definitions, Representation and Types, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 10

10. Set Operations

Learning Objectives

Perform union, intersection, difference, symmetric difference and complement.

1. Theoretical Concept

- $A \cup B$ contains elements in A or B.
- $A \cap B$ contains elements common to A and B.
- $A - B$ contains elements in A but not B.
- $A \oplus B$ contains elements in exactly one of the two sets.
- Complement depends on a specified universal set U.

2. Computer Science Connection

Set operations resemble database UNION, INTERSECTION and difference operations.

3. Python Implementation

```
U = set(range(1, 11))
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7}

print("Union:", A | B)
print("Intersection:", A & B)
print("A-B:", A - B)
print("Symmetric difference:", A ^ B)
print("Complement of A:", U - A)
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Find union and intersection of two sets by hand.
2. Repeat the problem in Python.
3. Using $U=\{1,...,20\}$, find the complement of the even numbers.
4. Verify one of De Morgan's set laws with Python.

6. Lesson Summary

Students learned the central ideas of Set Operations, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 11

11. Cartesian Product and Power Set

Learning Objectives

Construct Cartesian products and power sets and understand their size.

1. Theoretical Concept

- $A \times B$ is the set of ordered pairs (a,b) where $a \in A$ and $b \in B$.
- If $|A|=m$ and $|B|=n$, then $|A \times B|=mn$.
- The power set $P(A)$ is the set of all subsets of A .
- If $|A|=n$, then $|P(A)|=2^n$.

2. Computer Science Connection

Cartesian products appear in databases and combinations of attributes; power sets model all possible subsets.

3. Python Implementation

```
from itertools import product, combinations

A = {1, 2}
B = {"x", "y", "z"}

cartesian = list(product(A, B))
print(cartesian)

def powerset(s):
    s = list(s)
    return [set(c) for r in range(len(s)+1) for c in combinations(s, r)]

print(powerset(A))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Find $A \times B$ for two small sets.
2. Verify $|A \times B|=|A||B|$ in Python.
3. Generate the power set of $\{a,b,c\}$.
4. Why does a set with n elements have 2^n subsets?

6. Lesson Summary

Students learned the central ideas of Cartesian Product and Power Set, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 12

12. Sets, Venn Diagrams and Counting Applications

Learning Objectives

Apply set operations to real computing-style data and solve inclusion questions.

1. Theoretical Concept

- Set diagrams help visualize overlap among groups.
- For two finite sets, $|AB|=|A|+|B|-|A\cap B|$.
- The same idea is useful for database queries, duplicate removal and membership analysis.

2. Computer Science Connection

Set-based analysis is useful for course enrollment, permissions, duplicate removal and database queries.

3. Python Implementation

```
python_students = {"Ali", "Sara", "Hina", "Usman"}
math_students = {"Sara", "Hina", "Bilal"}

print("Both:", python_students & math_students)
print("Only Python:", python_students - math_students)
print("At least one:", python_students | math_students)
print("Neither (from class):", {"Ali", "Sara", "Hina", "Usman", "Bilal", "Ayesha"} - (python_students | math_students))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Solve a two-set survey problem using the inclusion formula.
2. Create two sets of students and find their common members.
3. Write a program that reports students in exactly one course.
4. Design a three-set example and explain what information a Venn diagram shows.

6. Lesson Summary

Students learned the central ideas of Sets, Venn Diagrams and Counting Applications, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 13

13. Relations: Definition, Representation and Properties

Learning Objectives

Represent binary relations and test reflexive, symmetric, antisymmetric and transitive properties.

1. Theoretical Concept

- A relation from A to B is a subset of $A \times B$.
- A relation on A is a subset of $A \times A$.
- Reflexive: (a,a) belongs to R for every a.
- Symmetric: $(a,b)R$ implies $(b,a)R$.
- Antisymmetric: (a,b) and (b,a) imply $a=b$.
- Transitive: (a,b) and (b,c) imply (a,c) .

Relation matrix

For finite sets, a relation can be represented by a Boolean matrix M where $M[i][j]=1$ exactly when (a_i, a_j) belongs to the relation.

2. Computer Science Connection

Relations model links between records, prerequisites, social connections and database relationships.

3. Python Implementation

```
A = {1, 2, 3}
R = {(1,1), (2,2), (3,3), (1,2), (2,1)}

def is_reflexive(A, R):
    return all((a,a) in R for a in A)

def is_symmetric(R):
    return all((b,a) in R for a,b in R)

print("Reflexive:", is_reflexive(A, R))
print("Symmetric:", is_symmetric(R))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. List a relation on $\{1,2,3\}$.
2. Test it for reflexivity and symmetry by hand.
3. Complete the Python code for antisymmetry and transitivity.
4. Give one relation that is symmetric but not transitive.

6. Lesson Summary

Students learned the central ideas of Relations: Definition, Representation and Properties, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 14

14. Equivalence Relations and Partitions

Learning Objectives

Recognize equivalence relations and connect their equivalence classes with partitions.

1. Theoretical Concept

- An equivalence relation is reflexive, symmetric and transitive.
- An equivalence class $[a]$ contains all elements related to a .
- Equivalence classes form a partition: they are nonempty, pairwise disjoint and cover the set.

Partition idea

A partition divides a set into nonempty, non-overlapping blocks whose union is the whole set.

2. Computer Science Connection

Equivalence classes are useful for grouping objects that are considered identical under a rule.

3. Python Implementation

```
A = {0, 1, 2, 3, 4, 5}
# Same parity relation
R = {(a,b) for a in A for b in A if a % 2 == b % 2}

print("Number of pairs:", len(R))
print("Class of 0:", {b for a,b in R if a == 0})
print("Class of 1:", {b for a,b in R if a == 1})
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Show that 'same parity' is an equivalence relation.
2. Find equivalence classes modulo 3 on $\{0, \dots, 8\}$.
3. Generate those classes in Python.
4. Explain why equivalence classes do not overlap partially.

6. Lesson Summary

Students learned the central ideas of Equivalence Relations and Partitions, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 15

15. Functions and Mappings

Learning Objectives

Understand functions, domain, codomain, range, injective, surjective and bijective mappings.

1. Theoretical Concept

- A function $f:A \rightarrow B$ assigns exactly one element of B to each element of A .
- The domain is A ; the codomain is B ; the range is the set of actual outputs.
- Injective means different inputs have different outputs.
- Surjective means every codomain element is hit.
- Bijective means both injective and surjective.

2. Computer Science Connection

Functions model input-output behavior and are fundamental to programming.

3. Python Implementation

```
def f(x):  
    return 2*x + 1  
  
A = {0,1,2,3}  
image = {f(x) for x in A}  
print("Domain:", A)  
print("Range:", image)
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Determine whether $f(x)=2x+1$ is injective over integers.
2. For finite sets, test a mapping for injectivity using Python.
3. Give an example of a surjective but non-injective function.
4. Explain why a function cannot assign two outputs to one input.

6. Lesson Summary

Students learned the central ideas of Functions and Mappings, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 16

16. Composition, Inverse and Recursive Functions

Learning Objectives

Compute function composition and understand when an inverse function exists.

1. Theoretical Concept

- Composition $(fg)(x)=f(g(x))$.
- A function has an inverse function on its range when it is one-to-one.
- For a bijection $f:A \rightarrow B$, $f^{-1}:B \rightarrow A$ exists and reverses the mapping.

2. Computer Science Connection

Composition appears when software functions are chained; inverse mappings occur in encoding and decoding.

3. Python Implementation

```
def f(x): return 2*x + 3
def g(x): return x*x

x = 4
print("f(g(x)) =", f(g(x)))
print("g(f(x)) =", g(f(x)))

# A simple inverse of f(x)=2x+3
y = 11
print("f^-1(y) =", (y-3)/2)
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Find fg and gf for $f(x)=x+1$, $g(x)=2x$.
2. Find the inverse of $f(x)=3x-2$.
3. Verify $f(f^{-1}(y))=y$ for several values in Python.
4. Explain why a non-injective function generally has no function inverse.

6. Lesson Summary

Students learned the central ideas of Composition, Inverse and Recursive Functions, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 17

17. Sequences and Series

Learning Objectives

Represent sequences, compute terms and partial sums, and connect formulas to Python.

1. Theoretical Concept

- A sequence is an ordered list $a_1, a_2, \dots$ indexed by integers.
- An arithmetic sequence has constant difference d .
- A geometric sequence has constant ratio r .
- A series is the sum of terms of a sequence.

2. Computer Science Connection

Sequences model time-series data, algorithm steps and generated data.

3. Python Implementation

```
def arithmetic(a1, d, n):  
    return a1 + (n-1)*d  
  
def geometric(a1, r, n):  
    return a1 * r**(n-1)  
  
print([arithmetic(3, 2, n) for n in range(1, 6)])  
print(sum(arithmetic(3, 2, n) for n in range(1, 6)))  
print([geometric(2, 3, n) for n in range(1, 5)])
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Find the 20th term of an arithmetic sequence.
2. Write Python code for the first 15 geometric terms.
3. Compute the partial sum of the first n natural numbers.
4. Compare an arithmetic and geometric sequence in a small table.

6. Lesson Summary

Students learned the central ideas of Sequences and Series, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 18

18. Recurrence Relations

Learning Objectives

Define and solve simple recurrence relations and implement iterative and recursive versions.

1. Theoretical Concept

- A recurrence defines a term using earlier terms.
- Example: $a_n = a_{n-1} + 3$ with $a_1 = 2$.
- Recurrences are fundamental in algorithm analysis and recursive programming.

Recurrence vs explicit formula

A recurrence relates a term to earlier terms; an explicit formula gives the term directly in terms of n .

2. Computer Science Connection

Recurrences are used to describe recursive algorithms and analyze their behavior.

3. Python Implementation

```
def recurrence(n):  
    a = 2  
    for _ in range(2, n+1):  
        a = a + 3  
    return a  
  
print([recurrence(n) for n in range(1, 8)])
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Find the first six terms of $a_n = 2a_{n-1}$, $a_1 = 3$.
2. Implement the recurrence in Python.
3. Explain the difference between a recurrence and an explicit formula.
4. Solve $a_n = a_{n-1} + 5$, $a_1 = 1$.

6. Lesson Summary

Students learned the central ideas of Recurrence Relations, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 19

19. Counting Principles: Sum and Product Rules

Learning Objectives

Apply addition and multiplication principles to counting problems.

1. Theoretical Concept

- If choices are mutually exclusive, add their counts.
- If a task has successive independent stages, multiply the number of choices.
- These principles are the foundation of more advanced counting.

2. Computer Science Connection

Counting helps estimate possible passwords, configurations and test cases.

3. Python Implementation

```
# Number of three-character strings using A, B, C, D
letters = "ABCD"
count = len(letters)**3
print("Total:", count)

# Explicit generation
items = [a+b+c for a in letters for b in letters for c in letters]
print(len(items))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. How many ways can one choose one item from groups of 5 and 7?
2. How many 4-digit PINs are possible if repetition is allowed?
3. Generate all two-letter strings from ABCD.
4. Modify the program so repeated letters are not allowed.

6. Lesson Summary

Students learned the central ideas of Counting Principles: Sum and Product Rules, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 20

20. Permutations and Combinations

Learning Objectives

Distinguish ordered arrangements from unordered selections and compute both.

1. Theoretical Concept

- Permutation counts arrangements where order matters.
- Combination counts selections where order does not matter.
- $nPr = n! / (n-r)!$ and $nCr = n! / [r!(n-r)!]$.

2. Computer Science Connection

Permutations and combinations are fundamental in search spaces, scheduling and probability.

3. Python Implementation

```
import math

n, r = 6, 2
print("Permutation:", math.perm(n, r))
print("Combination:", math.comb(n, r))

# Manual formulas
print(math.factorial(n) // math.factorial(n-r))
print(math.factorial(n) // (math.factorial(r)*math.factorial(n-r)))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Calculate $8P3$ and $8C3$.
2. Explain why selecting a committee uses combinations.
3. Write a Python program that accepts n and r .
4. Give one example where changing order changes the outcome.

6. Lesson Summary

Students learned the central ideas of Permutations and Combinations, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 21

21. Binomial Coefficients and Binomial Theorem

Learning Objectives

Use binomial coefficients and expand small powers using Python.

1. Theoretical Concept

- The binomial theorem gives $(a+b)^n = \sum C(n,k)a^{n-k}b^k$.
- The coefficients in the expansion are binomial coefficients.
- Pascal's triangle is another way to generate these coefficients.

2. Computer Science Connection

Binomial coefficients occur in probability, combinatorial algorithms and polynomial expansion.

3. Python Implementation

```
import math

def binomial_coefficients(n):
    return [math.comb(n, k) for k in range(n+1)]

print(binomial_coefficients(5))

a, b, n = 2, 3, 4
value = sum(math.comb(n, k) * a**(n-k) * b**k for k in range(n+1))
print(value, (a+b)**n)
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Write the expansion of $(x+y)^4$.
2. Generate row 8 of Pascal's triangle.
3. Verify the binomial theorem numerically in Python.
4. Explain the meaning of $C(n,k)$ in the expansion.

6. Lesson Summary

Students learned the central ideas of Binomial Coefficients and Binomial Theorem, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 22

22. Inclusion–Exclusion Principle

Learning Objectives

Use inclusion–exclusion for two and three finite sets.

1. Theoretical Concept

- For two sets: $|A \cup B| = |A| + |B| - |A \cap B|$.
- For three sets, add the three single-set sizes, subtract pairwise intersections, then add the triple intersection.
- The subtraction prevents double counting.

2. Computer Science Connection

Inclusion-exclusion helps count overlapping database categories and search results.

3. Python Implementation

```
A = set(range(1, 11))
B = set(range(5, 16))
C = set(range(8, 13))

direct = len(A | B | C)
formula = (len(A)+len(B)+len(C)
           - len(A&B) - len(A&C) - len(B&C)
           + len(A&B&C))
print(direct, formula)
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Solve a two-set survey problem.
2. Solve a three-set counting problem using the formula.
3. Verify your answer with Python sets.
4. Explain exactly why pairwise intersections are subtracted.

6. Lesson Summary

Students learned the central ideas of Inclusion–Exclusion Principle, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 23

23. Pigeonhole Principle

Learning Objectives

Apply the pigeonhole principle to prove existence results and simulate simple cases.

1. Theoretical Concept

- If more than n objects are placed into n boxes, at least one box contains at least two objects.
- Generalized form: placing N objects into k boxes guarantees a box with at least $\lceil N/k \rceil$ objects.
- The principle is an existence argument; we do not need to identify the exact box beforehand.

2. Computer Science Connection

Pigeonhole reasoning supports collision arguments in hashing and guaranteed repetition.

3. Python Implementation

```
from collections import Counter

birth_months = [1,2,3,4,5,6,7,8,9,10,11,12,1,2]
counts = Counter(birth_months)
print(counts)
print("Maximum in one month:", max(counts.values()))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Explain why among 13 people at least two share a birth month.
2. Find the minimum number of students needed to guarantee 3 have the same month.
3. Simulate random assignments to 12 months.
4. State the generalized pigeonhole principle.

6. Lesson Summary

Students learned the central ideas of Pigeonhole Principle, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 24

24. Number Theory: Divisibility, Primes and GCD

Learning Objectives

Use divisibility rules, prime numbers, GCD and Euclidean algorithm.

1. Theoretical Concept

- a divides b if $b=ak$ for some integer k .
- A prime number greater than 1 has exactly two positive divisors.
- The Euclidean algorithm computes gcd efficiently using repeated remainders.

Euclidean algorithm

For $a > b > 0$, $\text{gcd}(a,b) = \text{gcd}(b, a \bmod b)$. Repeat until the remainder is zero; the last nonzero remainder is the GCD.

2. Computer Science Connection

Number theory underlies cryptography, hashing and modular algorithms.

3. Python Implementation

```
import math

print("gcd:", math.gcd(84, 30))

def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n**0.5)+1):
        if n % d == 0:
            return False
    return True

print([n for n in range(2, 30) if is_prime(n)])
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Find $\text{gcd}(252, 105)$ by Euclidean algorithm.
2. Write a Python prime checker.
3. Generate all primes below 100.
4. Explain why checking divisors only up to $\sqrt{n}$ is sufficient.

6. Lesson Summary

Students learned the central ideas of Number Theory: Divisibility, Primes and GCD, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 25

25. Modular Arithmetic and Applications

Learning Objectives

Perform modular calculations and understand congruence.

1. Theoretical Concept

- $a \equiv b \pmod{n}$ means n divides $a-b$.
- Modular arithmetic is useful in clocks, hashing, cryptography and cyclic processes.
- Python's `%` operator returns the remainder and `pow(a,b,m)` performs modular exponentiation efficiently.

Congruence properties

Congruent numbers can be added and multiplied while preserving the modulus relationship.

2. Computer Science Connection

Modular arithmetic is fundamental to cryptography, checksums and cyclic indexing.

3. Python Implementation

```
print(17 % 5)
print(17 % 12)
print((7 + 9) % 5)
print((7 * 9) % 5)
print(pow(3, 100, 7))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Compute several congruences by hand.
2. Verify that congruent numbers have the same remainder.
3. Use modular arithmetic to implement a simple Caesar shift.
4. Explain why modular arithmetic is useful in cryptography.

6. Lesson Summary

Students learned the central ideas of Modular Arithmetic and Applications, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 26

26. Algorithms: Searching and Sorting

Learning Objectives

Understand algorithmic steps and implement linear search, binary search and basic sorting.

1. Theoretical Concept

- An algorithm is a finite, unambiguous sequence of steps for solving a problem.
- Linear search checks items one by one.
- Binary search repeatedly halves a sorted search space.
- Sorting arranges data according to a chosen order.

Algorithm thinking

Emphasize input, output, correctness, termination and efficiency. Binary search requires sorted data.

2. Computer Science Connection

Searching and sorting are core algorithmic tasks and prepare students for data structures.

3. Python Implementation

```
def linear_search(A, target):  
    for i, value in enumerate(A):  
        if value == target:  
            return i  
    return -1
```

```
A = [7, 2, 9, 4, 1]  
print(linear_search(A, 9))  
print(sorted(A))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Implement linear search without using `index()`.
2. Write binary search for a sorted list.
3. Compare the number of comparisons for linear and binary search.
4. Sort a list using a simple algorithm such as bubble sort.

6. Lesson Summary

Students learned the central ideas of Algorithms: Searching and Sorting, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 27

27. Graph Theory: Basic Concepts

Learning Objectives

Define graphs, vertices, edges, degree, paths and cycles.

1. Theoretical Concept

- A graph $G=(V,E)$ consists of vertices V and edges E .
- A simple undirected graph has no loops or multiple edges.
- The degree of a vertex is the number of incident edges.
- A path is a sequence of connected vertices; a cycle starts and ends at the same vertex.

Handshaking theorem

For every finite undirected graph, the sum of all vertex degrees equals $2|E|$; hence the number of odd-degree vertices is even.

2. Computer Science Connection

Graphs model networks, roads, social links, communication systems and dependencies.

3. Python Implementation

```
V = {"A", "B", "C", "D"}
E = {("A", "B"), ("A", "C"), ("B", "D"), ("C", "D")}

degree = {v: 0 for v in V}
for u, v in E:
    degree[u] += 1
    degree[v] += 1

print(degree)
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Draw a graph with five vertices and six edges.
2. Calculate each vertex degree.
3. Represent the graph in Python.
4. Verify the handshaking theorem: sum of degrees = $2|E|$.

6. Lesson Summary

Students learned the central ideas of Graph Theory: Basic Concepts, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 28

28. Graph Representation, Paths and Traversal

Learning Objectives

Represent graphs with adjacency lists/matrices and perform BFS/DFS.

1. Theoretical Concept

- An adjacency list stores neighbors for each vertex.
- An adjacency matrix stores whether each pair of vertices is connected.
- Breadth-first search explores level by level; depth-first search explores as far as possible before backtracking.

BFS vs DFS

BFS uses a queue and explores level by level. DFS uses recursion or a stack and explores deeply before backtracking.

2. Computer Science Connection

Graph traversal is used in routing, web crawling, path finding and network analysis.

3. Python Implementation

```
from collections import deque
```

```
G = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"]
}
```

```
def bfs(G, start):
    seen = {start}
    q = deque([start])
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in G[u]:
            if v not in seen:
                seen.add(v)
                q.append(v)
    return order
```

```
print(bfs(G, "A"))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Create an adjacency matrix for a four-vertex graph.
2. Implement DFS recursively.
3. Use BFS to find the order in which vertices are visited.
4. Explain one practical use of BFS in computing.

6. Lesson Summary

Students learned the central ideas of Graph Representation, Paths and Traversal, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 29

29. Euler and Hamiltonian Graphs

Learning Objectives

Differentiate Euler trails/circuits from Hamiltonian paths/cycles and apply basic conditions.

1. Theoretical Concept

- An Euler trail uses every edge exactly once; an Euler circuit starts and ends at the same vertex.
- A Hamiltonian path visits every vertex exactly once; a Hamiltonian cycle returns to the starting vertex.
- For a connected undirected graph, an Euler circuit exists exactly when every vertex has even degree.
- Euler and Hamiltonian properties are different: one focuses on edges, the other on vertices.

2. Computer Science Connection

Euler ideas model route inspection; Hamiltonian ideas model visiting locations or tasks.

3. Python Implementation

```
def all_even(degrees):  
    return all(d % 2 == 0 for d in degrees.values())  
  
degrees = {"A":2, "B":4, "C":2, "D":2}  
print("Euler circuit condition:", all_even(degrees))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Determine whether several small graphs have Euler circuits.
2. Explain the difference between Euler and Hamiltonian concepts.
3. Write Python code to test the even-degree condition.
4. Give a real-world route-planning example related to Hamiltonian paths.

6. Lesson Summary

Students learned the central ideas of Euler and Hamiltonian Graphs, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 30

30. Graph Coloring and Planar Graphs

Learning Objectives

Understand proper vertex coloring and basic planar-graph ideas.

1. Theoretical Concept

- A proper coloring assigns colors to vertices so adjacent vertices have different colors.
- The chromatic number is the minimum number of colors needed.
- A planar graph can be drawn in the plane without edge crossings.
- Coloring appears in scheduling, frequency assignment and register allocation.

2. Computer Science Connection

Coloring can represent scheduling conflicts, frequency assignment and compiler register allocation.

3. Python Implementation

```
G = {
    1: {2,3},
    2: {1,3},
    3: {1,2,4},
    4: {3}
}

color = {}
for v in G:
    forbidden = {color[u] for u in G[v] if u in color}
    c = 0
    while c in forbidden:
        c += 1
    color[v] = c

print(color)
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Color a triangle by hand.
2. Find a proper coloring of a path with five vertices.
3. Run the greedy Python coloring on a graph.
4. Explain why graph coloring can model exam scheduling.

6. Lesson Summary

Students learned the central ideas of Graph Coloring and Planar Graphs, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 31

31. Trees, Rooted Trees and Traversals

Learning Objectives

Understand trees, rooted trees, leaves and common traversal methods.

1. Theoretical Concept

- A tree is a connected undirected graph with no cycles.
- A tree with n vertices has exactly $n-1$ edges.
- A rooted tree has a designated root and parent-child relationships.
- Common traversals include preorder, inorder and postorder for binary trees; BFS is also used for level-order traversal.

Tree facts

A tree with n vertices has $n-1$ edges. Between any two vertices there is exactly one simple path.

2. Computer Science Connection

Trees appear in file systems, syntax trees, search trees, databases and hierarchical data.

3. Python Implementation

```
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right

root = Node(10, Node(5), Node(15))

def preorder(node):
    if node is None:
        return []
    return [node.value] + preorder(node.left) + preorder(node.right)

print(preorder(root))
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Draw a rooted tree with at least 7 nodes.
2. Verify the $n-1$ edge rule on several trees.
3. Implement inorder and postorder traversal.
4. Explain where tree structures are used in computer science.

6. Lesson Summary

Students learned the central ideas of Trees, Rooted Trees and Traversals, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Lesson 32

32. Comprehensive Review and Integrated Python Applications

Learning Objectives

Integrate the course concepts into small computing problems and prepare for assessment.

1. Theoretical Concept

- Discrete mathematics provides models for logic, data organization, algorithms and relationships.
- A strong solution should include mathematical reasoning and, where appropriate, a correct computational implementation.
- Python experiments are evidence for finite examples; proofs are required for universal mathematical claims.

2. Computer Science Connection

The integrated problems demonstrate how mathematics, algorithms and Python work together.

3. Python Implementation

```
# Integrated example: count even numbers, remove duplicates,
# and report their positions.
data = [4, 7, 4, 9, 2, 7, 8, 2]
unique = set(data)
evens = {x for x in unique if x % 2 == 0}

print("Unique:", unique)
print("Even values:", evens)
print("Positions of even values:",
      [i for i,x in enumerate(data) if x % 2 == 0])
```

4. Worked Example / Classroom Explanation

First solve a small example by hand. Then run the Python code and compare the result. Ask students to predict the output before execution, change one input, and explain why the mathematical result changes. For proof-oriented lessons, label the program as an experiment on tested cases rather than a universal proof.

5. Practice Questions

1. Create a mini project combining sets, functions and counting.
2. Build a truth-table generator for a chosen compound proposition.
3. Build a graph and implement BFS and DFS.
4. Prepare a short report explaining one discrete concept and its Python application.
5. Review all 32 lessons and identify one concept that still needs practice.

6. Lesson Summary

Students learned the central ideas of Comprehensive Review and Integrated Python Applications, connected them to a computing application, and practiced the concept through Python.

7. Quick Check

Students should be able to state the main definition, solve a basic mathematical example, explain the corresponding Python operation, and modify the code for a new example.

Teacher's Implementation Guide

Recommended 50–60 minute lesson rhythm

- 5–8 minutes: previous-lesson review and warm-up question.
- 15–20 minutes: mathematical theory and worked examples.
- 10–15 minutes: Python demonstration; students predict output before running it.
- 10 minutes: guided student practice.
- 5 minutes: exit question / quick check.

Recommended LMS upload format

- Upload each lesson separately using the lesson number and topic as the title.
- Keep theory and Python in the same lesson.
- Place practice questions at the end.
- For selected lessons, upload a separate solution file after submission.
- Keep this master PDF for teacher reference and use individual lesson sections/files for LMS delivery.

Python Environment

Students may use standard Python 3, including a local installation or Jupyter Notebook. The examples use the Python standard library unless otherwise stated.

Reference and Alignment

The course structure is aligned with the HEC Computing Disciplines curriculum for Discrete Structure. Major HEC topics represented here include logic and reasoning, proofs, sets, relations, functions, recurrence, number theory, counting, algorithms, graphs and trees, with computing applications throughout.

End of Notes

Discrete Structure + Python

Prepared by Abdul Salam